



A. MỤC TIÊU:

- Giới thiệu về các nguồn tài nguyên trên thiết bị
- Sử dụng tài nguyên của thiết bị: quản lý quyền của ứng dụng, truy cập vào danh bạ, tích hợp camera vào ứng dụng.
- Ứng dụng truy cập dữ liệu danh bạ và tin nhắn trên thiết bị điện thoại.

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SV:

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

1.1. Application Sandbox

Application Sandbox, hay còn gọi là App Sandbox hoặc Sandbox, là một cơ chế "cô lập" mà hệ điều hành di động (iOS và Android) áp dụng cho mỗi ứng dụng. Cơ chế này bắt buộc mỗi ứng dụng chạy trong một UIU (user ID) riêng biệt trên hệ điều hành. Ứng dụng không thể tự do truy cập các file, dữ liệu hoặc cài đặt của các ứng dụng khác trong hệ thống, trừ khi hệ điều hành cho phép (được cấp quyền).

Đây là một trong những tính năng bảo mật cốt lõi của hệ điều hành di động hiện đại. Tính năng này giúp:

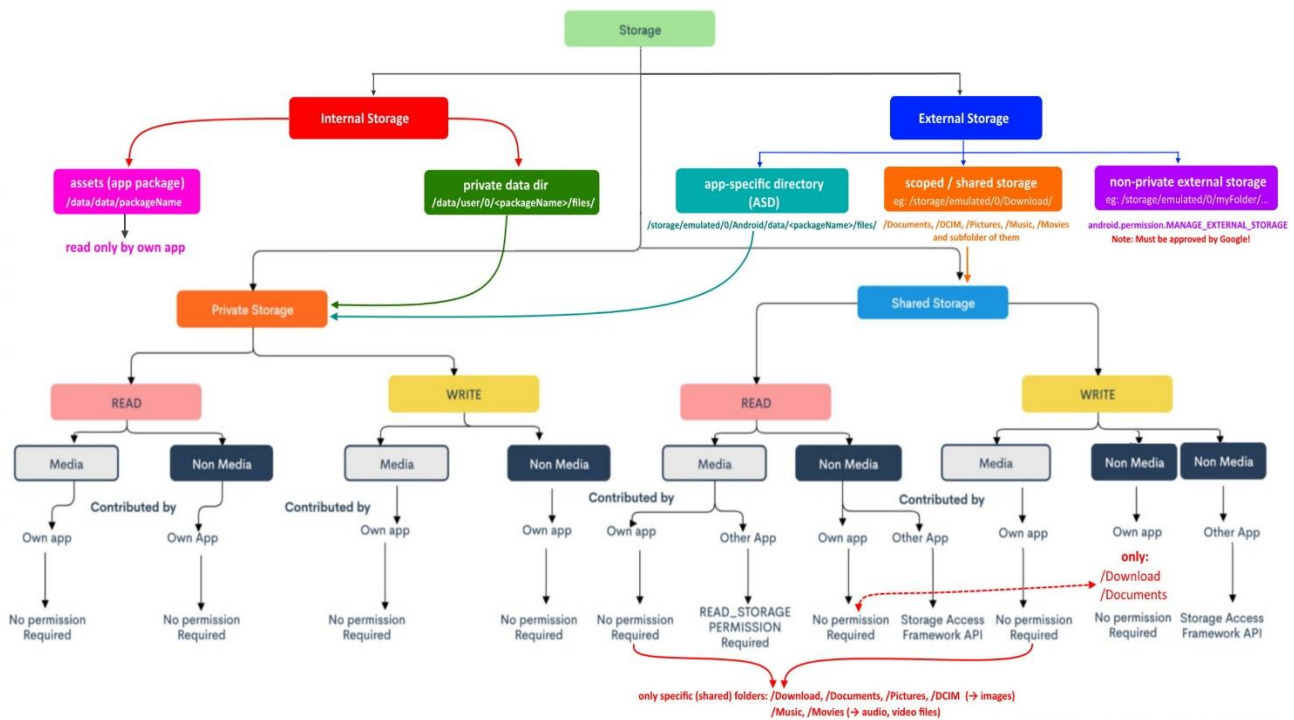
- Ngăn các ứng dụng độc hại đọc/ghi dữ liệu của ứng dụng khác (ví dụ: app A không đọc được dữ liệu của app B).
- Ngăn gây sự cố toàn hệ thống nếu có lỗi.
- Bảo vệ quyền riêng tư người dùng.
- Làm cho việc gỡ cài đặt ứng dụng sạch sẽ (xóa hết dữ liệu liên quan).

Như vậy, ứng dụng muốn truy xuất ngoài sandbox thì phải xin cấp quyền (permission) hoặc dùng API hệ thống.

1.2. Phân loại dữ liệu trong ứng dụng di động

Dữ liệu trong ứng dụng di động thường được phân loại theo nguồn lưu trữ và cách ứng dụng truy cập. Mỗi loại dữ liệu có đặc điểm, quyền truy cập và mục đích sử dụng khác nhau. Thường có 4 loại sau :

- Dữ liệu cục bộ
- Dữ liệu từ máy chủ
- Dữ liệu trạng thái ứng dụng
- Dữ liệu cache



<https://community.kodular.io/t/some-basics-on-android-storage-system/69074/20>

1.2.1. Dữ liệu cục bộ - Local Data

- **Khái niệm :** là dữ liệu được lưu trực tiếp trên thiết bị người dùng. Ứng dụng có thể truy cập ngay cả khi không có Internet. Dữ liệu được lưu trong bộ nhớ của thiết bị và vẫn tồn tại khi đóng ứng dụng hoặc khi khởi động lại ứng dụng. Tuy nhiên, dữ liệu sẽ mất khi gỡ cài đặt ứng dụng hoặc xóa dữ liệu ứng dụng.
- **Quyền truy cập :**
 - + Chỉ ứng dụng đó mới truy cập được.
 - + Không cần internet.
 - + Lưu trong bộ nhớ thiết bị hoặc database cục bộ. Dữ liệu này không cho ứng dụng khác truy cập vào.
- **Áp dụng** trong trường hợp dữ liệu cần truy cập nhanh, không cần đồng bộ với server ngay, mang tính cá nhân của người dùng.
- **Dạng lưu trữ :**

(1) **SharedPreferences:** lưu dữ liệu cục bộ dạng key-value, lưu các thông tin nhỏ, cấu hình hoặc trạng thái của ứng dụng.

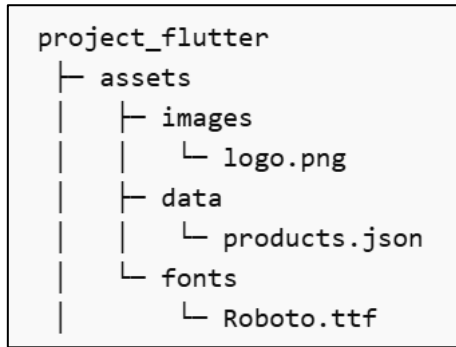
Ví dụ: lưu thông tin tài khoản đăng nhập cho các lần sau lần đăng nhập đầu tiên.

(2) **Local Database (SQLite):** lưu trữ dữ liệu có cấu trúc.

Ví dụ: cơ sở dữ liệu dạng table có liên kết lưu dữ liệu cục bộ, như danh sách chi tiêu trong ứng dụng quản lý chi tiêu cá nhân.

(3) **Dữ liệu cục bộ tĩnh (Static Local Data / App Bundled Data):** dữ liệu được đóng gói sẵn trong ứng dụng khi build, được cài đặt cùng với ứng dụng trên thiết bị. Dữ liệu này chỉ có quyền đọc, không cần truy cập internet.

Ví dụ: các dữ liệu trong thư mục assets như:



(4) File storage: là phương pháp lưu dữ liệu bằng cách ghi trực tiếp vào các file trong bộ nhớ thiết bị, như: văn bản, hình ảnh, âm thanh... Ứng dụng có thể đọc/ghi/xóa dữ liệu trên các file này. Dung lượng file có thể lớn, truy cập file không cần internet.

Lưu ý: Dữ liệu dạng này có thể cho ứng dụng khác truy cập, phụ thuộc vào vị trí lưu trữ của file trong hệ điều hành. Cụ thể như sau:

- Nếu file lưu trong bộ nhớ riêng của ứng dụng (**internal storage**) thì chỉ ứng dụng đó mới được truy cập file. Khi gỡ app dữ liệu sẽ mất

Trong android, lưu tại thư mục: */data/data/com.example.app/*

- Nếu file lưu trong bộ nhớ chia sẻ (**external file**) chung của thiết bị thì ứng dụng khác có thể truy cập. Khi gỡ app, dữ liệu có thể còn.

Trong android, lưu tại thư mục:

/storage/emulated/0/Download

/storage/emulated/0/Pictures

1.2.2. Dữ liệu từ máy chủ - Server data

Là dữ liệu được lưu trên máy server. Ứng dụng cần có internet mới truy cập được thông qua API. Dữ liệu trả về dạng xml, json.

1.2.3. Dữ liệu trạng thái ứng dụng – State data

- Là dữ liệu tạm thời trong bộ nhớ RAM khi ứng dụng đang chạy.
- Trong Flutter, quản lý trạng thái là cơ chế giúp quản lý dữ liệu của ứng dụng và cập nhật giao diện (UI) khi dữ liệu thay đổi.
- Các thư viện quản lý state trong Flutter: Provider, Bloc, Riverpod, GetX

Ví dụ: khi dữ liệu giỏ hàng thay đổi → UI hiển thị số lượng sản phẩm phải cập nhật;
khi API trả dữ liệu → danh sách sản phẩm phải hiển thị lại

1.2.4. Dữ liệu cache - Cache Data

- Là dữ liệu **được lưu tạm để tăng tốc truy cập**, thường lấy từ server rồi lưu lại trên bộ nhớ thiết bị, không cần truy cập internet.
- **Quyền truy cập:**
 - + lưu local

- + có thể bị xóa khi cần
- + thường có thời gian hết hạn
- **Khi nào nên dùng: dữ liệu có tính chất**
 - + truy cập thường xuyên
 - + không cần luôn chính xác tuyệt đối
 - + cần tăng tốc tải dữ liệu
- Flutter có các thư viện cache như: `cached_network_image`, `hive`, `sqflite`.

Ví dụ : ảnh sản phẩm, danh sách bài viết,...

1.3. Sử dụng nguồn tài nguyên của thiết bị

Trong các ứng dụng di động, việc truy cập hình ảnh, audio, video từ thư viện lưu trữ đa phương tiện trên thiết bị (gallery) hoặc trực tiếp từ camera là một tính năng phổ biến, ví dụ : chỉnh sửa ảnh, xác thực khuôn mặt, gửi ảnh/video qua các mạng xã hội, xem ảnh, nghe nhạc, xem video...

a/ Truy cập image, video từ gallery hoặc camera

- Để thực hiện truy cập image từ gallery hoặc camera, Flutter cung cấp các gói thư viện :
 - **image_picker**: Cho phép chọn ảnh/video từ gallery hoặc chụp ảnh/quay video từ camera.
 - **permission_handler**: quản lý quyền truy cập (camera, bộ nhớ).
 - **video_player**: hiển thị và phát video
- Các khái niệm liên quan :
 - **Gallery**: thư viện lưu trữ đa phương tiện trên thiết bị (ảnh, video).
 - **Camera**: thiết bị phần cứng để chụp ảnh hoặc quay video trực tiếp.
 - **Permissions** : quyền truy cập
- Android: yêu cầu quyền CAMERA, STORAGE.
- iOS: yêu cầu quyền NSCameraUsageDescription, NSPhotoLibraryUsageDescription trong file Info.plist.
- **File**: Hình ảnh/video được trả về dưới dạng file để hiển thị hoặc xử lý

b/ Cách sử dụng image_picker

- Mục đích: chọn ảnh/video từ gallery hoặc chụp/quay trực tiếp từ camera.
- Thêm **image_picker** vào **pubspec.yaml**
- Tạo instance của **ImagePicker**:

```
final ImagePicker _picker = ImagePicker();
```

- Chọn media:

- Ảnh từ gallery: `_picker.pickImage(source: ImageSource.gallery)`.
- Ảnh từ camera: `_picker.pickImage(source: ImageSource.camera)`.
- Video từ gallery: `_picker.pickVideo(source: ImageSource.gallery)`.
- Video từ camera: `_picker.pickVideo(source: ImageSource.camera)`.

- **Xử lý kết quả:** nhận XFile và chuyển thành File để hiển thị hoặc xử lý

```
XFile? file = await _picker.pickImage(source: ImageSource.gallery);
if (file != null) File imageFile = File(file.path);
```

c/ Cách sử dụng permission_handler

- Mục đích: quản lý quyền truy cập (camera, storage, microphone) trên Android/iOS.
- Thêm package: thêm **permission_handler** vào pubspec.yaml
- Cấu hình quyền trong Android hoặc iOS (tùy theo thực thi trên nền tảng nào).
Ví dụ trong Android: AndroidManifest.xml

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

- Kiểm tra quyền trước khi gọi image_picker:

```
if (await Permission.camera.isDenied) {
  await Permission.camera.request();
}
```

d/ Sử dụng video_player

- Mục đích: Phát video từ file hoặc URL.
- Thêm video_player vào pubspec.yaml
- Khởi tạo controller

```
VideoPlayerController _controller =
VideoPlayerController.file(File('path/to/video.mp4'));
await _controller.initialize();
```

- Phát video:
 - Hiển thị: Dùng widget **VideoPlayer(_controller)**.
 - Điều khiển: **_controller.play()**, **_controller.pause()**.
- Xóa khỏi bộ nhớ dữ liệu lưu trước đó (nếu có) để giải phóng tài nguyên

```
@override
void dispose() {
  _controller.dispose();
  super.dispose();
}
```

1.4. Truy cập dữ liệu tin nhắn và danh bạ trên thiết bị

Trong lập trình di động, hệ thống cho phép các ứng dụng được truy cập vào các dữ liệu danh bạ điện thoại (contacts) hoặc các tin nhắn SMS. Các dữ liệu hệ thống này lưu trong Content Provider (trên Android) và Contacts Framework (trên iOS, iOS giới hạn đọc tin nhắn bởi Apple). Các dữ liệu này muốn truy cập phải yêu cầu người dùng cấp quyền, tuân thủ quy định bảo mật của hệ thống.

a/ Truy cập tin nhắn SMS

- Mục đích : truy cập danh sách nội dung tin nhắn sms trên điện thoại lưu trong Content Provider. Chúng ta có thể truy cập được nội dung, số điện thoại gửi, thời gian, loại tin nhắn (gửi/nhận),...
- Thêm các package : **permission_handler** và **another_telephony**

b/ Truy cập danh bạ điện thoại

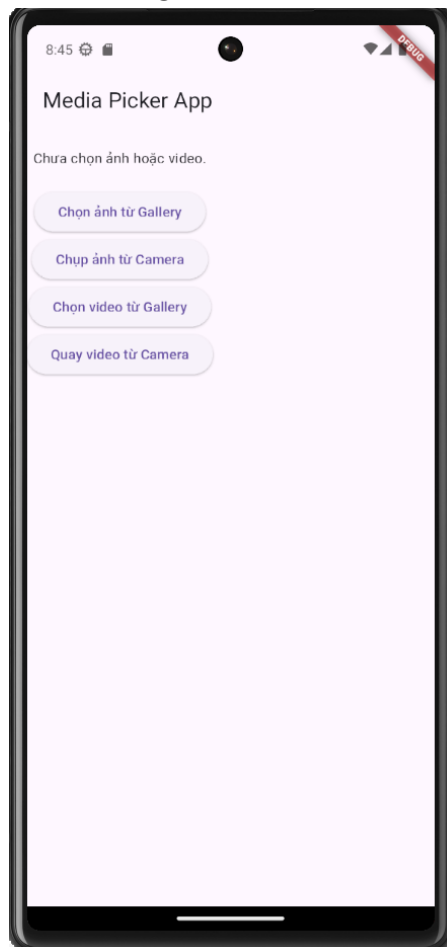
- Mục đích : truy cập, đọc, thêm, sửa, hoặc xóa thông tin danh bạ trên thiết bị người dùng. Ứng dụng trong xây dựng ứng dụng quản lý danh bạ, tích hợp danh bạ vào ứng dụng nhắn tin, gọi điện, hoặc đồng bộ danh bạ với dịch vụ đám mây.
- Thêm các package : **permission_handler** và **flutter_contacts_service**

Các package thư viện được cập nhật thường xuyên, cho nên người lập trình cần kiểm tra các phiên bản mới tại <https://pub.dev/packages>

2. Bài tập hướng dẫn

Bài tập 1. Tạo ứng dụng có các chức năng :

- Chọn ảnh từ gallery.
- Chụp ảnh từ camera.
- Chọn video từ gallery.
- Quay video từ camera.
- Hiện thị ảnh hoặc phát video trên giao diện.



Hướng dẫn :

Bước 1 : Cài đặt các package

- Tạo project mới dạng Empty Application.
- Mở file **pubspec.yaml** và thêm vào **dependencies** : (lưu ý các package phải ngang cấp với flutter package)

```
dependencies:  
  flutter:  
    sdk: flutter  
  image_picker: ^0.8.7+4  
  permission_handler: ^11.2.0  
  video_player: ^2.5.1
```

- Chạy lệnh để cài đặt các package trong terminal :

```
DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS  
  
PS E:\05.WORKSPACE\Flutter\demo_multimedia> flutter pub get  
Resolving dependencies...  
Downloading packages...  
  async 2.12.0 (2.13.0 available)  
  fake_async 1.3.2 (1.3.3 available)  
  leak_tracker 10.0.8 (10.0.9 available)  
  material_color_utilities 0.11.1 (0.12.0 available)  
  vm_service 14.3.1 (15.0.0 available)  
Got dependencies!  
5 packages have newer versions incompatible with dependency constraints.  
Try `flutter pub outdated` for more information.  
PS E:\05.WORKSPACE\Flutter\demo_multimedia> █
```

Lưu ý cách sửa lỗi khi các package không cài đặt được:

```
< image_picker_for_web 2.2.0 (was 3.0.6) (3.0.6 available)  
leak_tracker 10.0.8 (10.0.9 available)  
material_color_utilities 0.11.1 (0.12.0 available)  
< mime 1.0.6 (was 2.0.0) (2.0.0 available)  
permission_handler_android 12.1.0 (13.0.1 available)  
vm_service 14.3.1 (15.0.0 available)  
Changed 3 dependencies!  
9 packages have newer versions incompatible with dependency constraints.  
Try `flutter pub outdated` for more information.  
Building with plugins requires symlink support.  
Please enable Developer Mode in your system settings. Run  
start ms-settings:developers  
to open settings.  
exit code 1
```

Cách mở Developer Mode :

Cách thực hiện:

1. Mở Developer Mode Settings:

- o Chạy lệnh sau trong **Command Prompt** hoặc **Run (Windows + R)**:

```
start ms-settings:developers
```

- o Lệnh này sẽ mở trang **Developer Settings** trong Windows.

2. Bật Developer Mode:

- o Trong cửa sổ **Developer Settings**, chọn **Developer Mode**.
- o Xác nhận nếu có thông báo yêu cầu quyền.

3. Khởi động lại IDE (nếu cần):

- o Sau khi bật Developer Mode, khởi động lại Visual Studio Code hoặc bất kỳ IDE nào bạn đang sử dụng để đảm bảo thay đổi được áp dụng.

Lưu ý:

- Developer Mode không bắt buộc cho mọi dự án Flutter, nhưng nó cần thiết nếu bạn đang làm việc với các tính năng nâng cao hoặc cần quyền truy cập đặc biệt trên hệ thống.

Bước 2 : Cấu hình quyền truy cập :

a. Đối với Android

- Mở file *android/app/src/main/AndroidManifest.xml* và thêm quyền:

```
android > app > src > main > AndroidManifest.xml
1  <manifest xmlns:android="http://schemas.android.com/apk/res/android">
2
3      <!-- Khai báo các quyền ứng dụng yêu cầu -->
4      <uses-permission android:name='android.permission.CAMERA' />
5      <uses-permission android:name='android.permission.WRITE_EXTERNAL_STORAGE' />
6      <uses-permission android:name='android.permission.READ_EXTERNAL_STORAGE' />
7      <uses-permission android:name='android.permission.RECORD_AUDIO' />
8
9      <application
10         android:label="demo_multimedia"
```

Lưu ý : Sau khi thêm quyền vào file *AndroidManifest.xml*, nếu thực thi chương trình bị báo lỗi file chưa cập nhật thì có thể thực hiện các bước sau để khắc phục: chạy lần lượt các lệnh sau trong terminal

- Xóa bộ nhớ cache và các tệp build cũ để đảm bảo không có lỗi từ các tệp cũ:
`flutter clean`
- Cài đặt lại các gói flutter: `flutter pub get`
- Chạy ứng dụng trên thiết bị hoặc trình giả lập: `flutter run`

b. Đối với iOS

- Mở file *ios/Runner/Info.plist* và thêm

```
! pubspec.yaml • AndroidManifest.xml • Info.plist •
ios > Runner > Info.plist
4  <dict>
36  </array>
37  <key>UISupportedInterfaceOrientations~ipad</key>
38  <array>
39      <string>UIInterfaceOrientationPortrait</string>
40      <string>UIInterfaceOrientationPortraitUpsideDown</string>
41      <string>UIInterfaceOrientationLandscapeLeft</string>
42      <string>UIInterfaceOrientationLandscapeRight</string>
43  </array>
44  <key>CADisableMinimumFrameDurationOnPhone</key>
45  <true/>
46  <key>UIApplicationSupportsIndirectInputEvents</key>
47  <true/>
48
49  <key>NSCameraUsageDescription</key>
50  <string>Ứng dụng cần truy cập camera để chụp ảnh và quay video.</string>
51  <key>NSPhotoLibraryUsageDescription</key>
52  <string>Ứng dụng cần truy cập thư viện để chọn ảnh và video.</string>
53  <key>NSMicrophoneUsageDescription</key>
54  <string>Ứng dụng cần truy cập micro để quay video có âm thanh.</string>
55
56  </dict>
57  </plist>
58
```

Bước 3 : Thiết kế giao diện và xử lý

- Tạo file *lib/main.dart* có class *MediaPickerApp* như sau :

```
import 'dart:io';
import 'package:flutter/material.dart';
```

```

import 'package:image_picker/image_picker.dart';
import 'package:permission_handler/permission_handler.dart';
import 'package:video_player/video_player.dart';
import 'media_picker_home.dart';

void main() {
  runApp(MediaPickerApp());
}

class MediaPickerApp extends StatelessWidget {
  const MediaPickerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Media Picker App',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: MediaPickerHome(),
    );
  }
}

```

- Tạo file *lib/media_picker_home.dart* chứa class `MediaPickerHome` như sau :

```

import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import 'package:permission_handler/permission_handler.dart';
import 'package:video_player/video_player.dart';

class MediaPickerHome extends StatefulWidget {
  @override
  _MediaPickerHomeState createState() => _MediaPickerHomeState();
}

class _MediaPickerHomeState extends State<MediaPickerHome> {
  File? _mediaFile; //lưu trữ file media (image hoặc video)
  VideoPlayerController? _videoController; // điều khiển phát video
  final ImagePicker _picker =
    ImagePicker(); // khởi tạo ImagePicker để chọn ảnh hoặc video

  //Kiểm tra và yêu cầu quyền truy cập
  Future<void> _requestPermission(Permission permission) async {
    if (await permission
      .isDenied) //Kiểm tra xem quyền cụ thể (ví dụ: quyền truy cập bộ nhớ,
camera, v.v.) có bị từ chối hay không.
    {
      await permission
        .request(); //Nếu quyền bị từ chối, phương thức này sẽ hiển thị hộp
thoại yêu cầu người dùng cấp quyền.
    }
  }
}

```

```

// Chọn ảnh hoặc video từ gallery
Future<void> _pickMedia(ImageSource source, bool isVideo) async {
  await _requestPermission(
    isVideo ? Permission.storage : Permission.photos,
  ); // yêu cầu quyền truy cập bộ nhớ hoặc ảnh

  //thực hiện việc chọn một tệp media (ảnh hoặc video) từ nguồn được chỉ
  định (source).
  //Nó sử dụng thư viện image_picker để chọn ảnh hoặc video. Nếu không chọn
  được ảnh
  //(pickImage trả về null), nó sẽ thử chọn video bằng phương thức
  pickVideo.
  final XFile? pickedFile =
    await _picker.pickImage(
      source: source,
      imageQuality:
        100, // nếu bỏ các tham số này thì ảnh được hiển thị theo kích
thước gốc
      maxWidth: 1920,
      maxHeight: 1080,
    ) ??
    await _picker.pickVideo(
      source: source,
    ); // chọn ảnh hoặc video từ nguồn (camera hoặc thư viện)

  if (pickedFile != null) {
    setState(() {
      _mediaFile = File(pickedFile.path); // lưu trữ file media đã chọn
      if (_mediaFile!.path.endsWith('.mp4')) {
        // nếu file là video
        _videoController
          ?.dispose(); // giải phóng bộ nhớ của video controller trước đó
(nếu có)
        _videoController = VideoPlayerController.file(
          _mediaFile!,
        ); // tạo video controller mới với file video đã chọn
        _videoController!.initialize().then((_) {
          setState(
            () {},
          ); // cập nhật giao diện sau khi video controller được khởi tạo
          _videoController!.play(); // phát video
        });
      } else {
        _videoController
          ?.dispose(); // giải phóng bộ nhớ của video controller trước đó
(nếu có)
        _videoController =
          null; // đặt video controller về null nếu không phải là video
      }
    });
  } else {
    ScaffoldMessenger.of(

```

```

        context,
    ).showSnackBar(SnackBar(content: Text('No media selected')));
}
}

//Chụp ảnh hoặc quay video từ camera
Future<void> _captureMedia(bool isVideo) async {
    await _requestPermission(
        Permission.camera,
    ); // yêu cầu quyền truy cập camera
    if (isVideo) {
        await _requestPermission(
            Permission.microphone,
        ); // yêu cầu quyền truy cập microphone nếu là video
    }

    final XFile? capturedFile =
        isVideo
            ? await _picker.pickVideo(
                source: ImageSource.camera,
            ) // chọn video từ camera
            : await _picker.pickImage(
                source: ImageSource.camera,
            ); // chọn ảnh từ camera

    if (capturedFile != null) {
        setState(() {
            _mediaFile = File(capturedFile.path); // lưu trữ file media đã chọn
            if (isVideo) {
                _videoController
                    ?.dispose(); // giải phóng bộ nhớ của video controller trước đó
                (nếu có)
                _videoController = VideoPlayerController.file(
                    _mediaFile!,
                ); // tạo video controller mới với file video đã chọn
                _videoController!.initialize().then((_) {
                    setState(
                        () {},
                    ); // cập nhật giao diện sau khi video controller được khởi tạo
                    _videoController!.play(); // phát video
                });
            } else {
                _videoController
                    ?.dispose(); // giải phóng bộ nhớ của video controller trước đó
                (nếu có)
                _videoController =
                    null; // đặt video controller về null nếu không phải là video
            }
        });
    } else {
        ScaffoldMessenger.of(
            context,

```

```

        ).showSnackBar(SnackBar(content: Text('No media captured')));
    }
}

@override
void dispose() {
    _videoController
        ?.dispose(); // giải phóng bộ nhớ của video controller khi widget bị
hủy
    super.dispose();
}

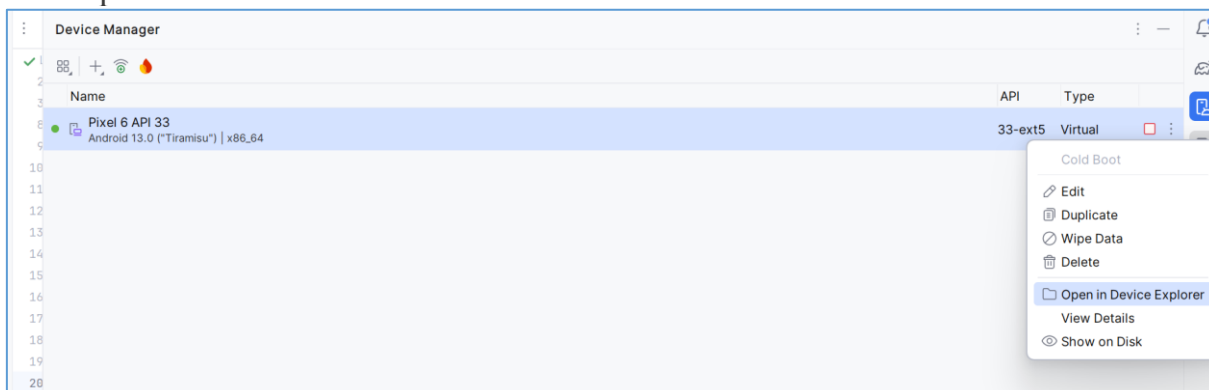
@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: Text('Media Picker App')),
        body: SingleChildScrollView(
            child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                    SizedBox(height: 30, width: 30),
                    _mediaFile == null
                        ? Text('Chưa chọn ảnh hoặc video.')
                        : _videoController != null &&
                          _videoController!.value.isInitialized
                        ? AspectRatio(
                            aspectRatio: _videoController!.value.aspectRatio,
                            child: VideoPlayer(_videoController!),
                        )
                        : Image.file(_mediaFile!, height: 300),
                    SizedBox(height: 20),
                    ElevatedButton(
                        onPressed: () => _pickMedia(ImageSource.gallery, false),
                        child: Text('Chọn ảnh từ Gallery'),
                    ),
                    ElevatedButton(
                        onPressed: () => _captureMedia(false),
                        child: Text('Chụp ảnh từ Camera'),
                    ),
                    ElevatedButton(
                        onPressed: () => _pickMedia(ImageSource.gallery, true),
                        child: Text('Chọn video từ Gallery'),
                    ),
                    ElevatedButton(
                        onPressed: () => _captureMedia(true),
                        child: Text('Quay video từ Camera'),
                    ),
                ],
            ),
        ),
    );
}

```

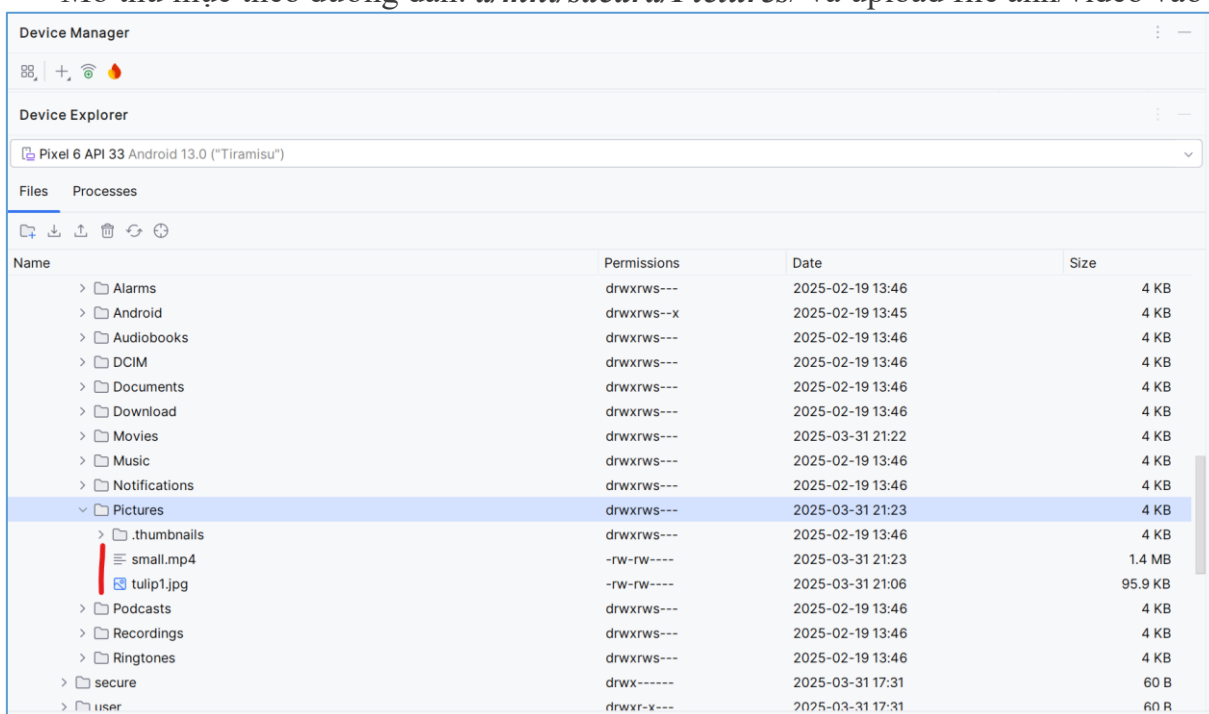
}

Để chuẩn bị cho dữ liệu test, chúng ta cần upload file ảnh và video (kích thước nhỏ) bỏ vào thư mục gallery của thiết bị. Thực hiện như sau:

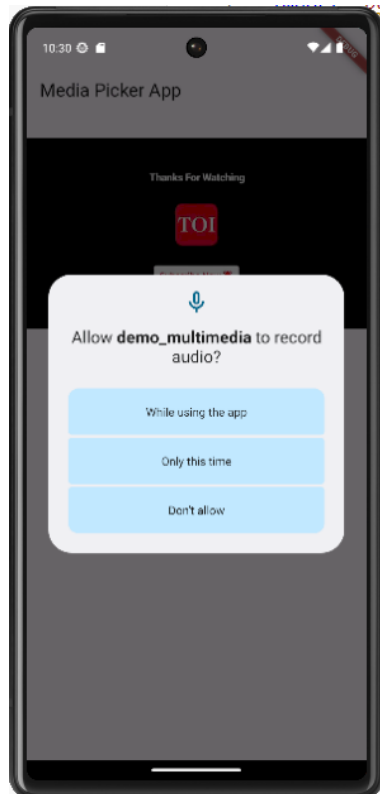
- Mở Android Studio: chọn thiết bị ảo để cài đặt ứng dụng, chọn Open in Device Explorer



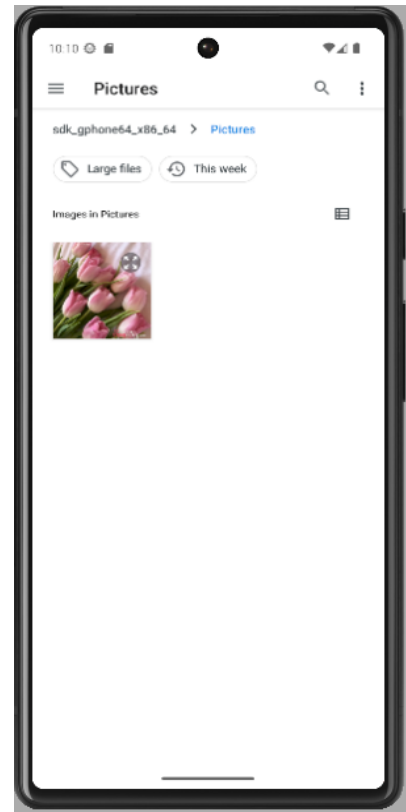
- Mở thư mục theo đường dẫn: `d/mnt/sdcard/Pictures/` và upload file ảnh/video vào



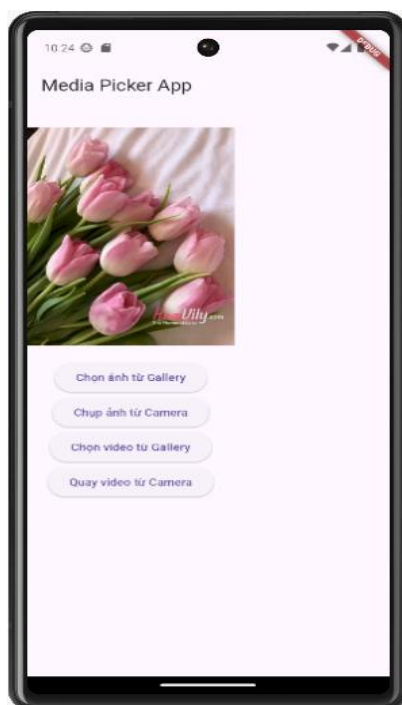
Khi ứng dụng thực thi từng chức năng:



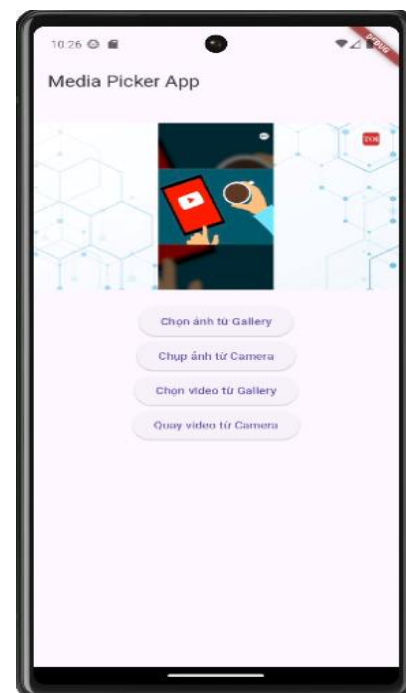
Hiện thị màn hình xin phép truy cập dữ liệu trên thiết bị



Chọn ảnh từ gallery



Hiện thị ảnh được chọn



Play video chọn từ Gallery



Hiện thị màn hình xin phép truy cập camera để chụp ảnh hoặc quay video



Màn hình giả lập mở camera trên máy ảo

Bài tập 2. Tạo ứng dụng “Photo Capture & Preview” cho phép

- Chụp ảnh từ camera hoặc chọn từ gallery.
- Hiện thị ảnh vừa chụp/chọn và cho phép xem trước toàn màn hình khi nhấn vào ảnh

Gợi ý:

- Tạo ứng dụng “Photo Capture & Preview”, cài đặt các package như bài tập 1.
- Cấu hình cấp quyền cho cho ứng dụng trên Android
- Viết mã code:

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import 'package:permission_handler/permission_handler.dart';

void main() {
  runApp(PhotoCaptureApp());
}

class PhotoCaptureApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Photo Capture & Preview',

```

```

        theme: ThemeData(primarySwatch: Colors.green),
        home: PhotoCaptureHome(),
    );
}
}

class PhotoCaptureHome extends StatefulWidget {
  @override
  _PhotoCaptureHomeState createState() => _PhotoCaptureHomeState();
}

class _PhotoCaptureHomeState extends State<PhotoCaptureHome> {
  File? _imageFile;
  final ImagePicker _picker = ImagePicker();

  // Yêu cầu quyền
  Future<void> _requestPermission(Permission permission) async {
    if (await permission.isDenied) {
      await permission.request();
    }
  }

  // Chọn ảnh từ gallery
  Future<void> _pickImageFromGallery() async {
    await _requestPermission(Permission.photos);
    final XFile? pickedFile = await _picker.pickImage(source:
    ImageSource.gallery);
    if (pickedFile != null) {
      setState(() {
        _imageFile = File(pickedFile.path);
      });
    }
  }

  // Chụp ảnh từ camera
  Future<void> _captureImageFromCamera() async {
    await _requestPermission(Permission.camera);
    final XFile? capturedFile = await _picker.pickImage(source:
    ImageSource.camera);
    if (capturedFile != null) {
      setState(() {
        _imageFile = File(capturedFile.path);
      });
    }
  }

  // Xem trước ảnh toàn màn hình
  void _showFullScreenPreview(BuildContext context) {
    if (_imageFile != null) {
      Navigator.push(
        context,
        MaterialPageRoute(

```

```

        builder: (context) => FullScreenImage(imageFile: _imageFile!),
      ),
    );
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text('Photo Capture & Preview')),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          _imageFile == null
            ? Text('Chưa có ảnh nào được chọn.')
            : GestureDetector(
                onTap: () => _showFullScreenPreview(context),
                child: Image.file(_imageFile!, height: 300),
              ),
          SizedBox(height: 20),
          ElevatedButton(
            onPressed: _pickImageFromGallery,
            child: Text('Chọn ảnh từ Gallery'),
          ),
          ElevatedButton(
            onPressed: _captureImageFromCamera,
            child: Text('Chụp ảnh từ Camera'),
          ),
        ],
      ),
    ),
  );
}

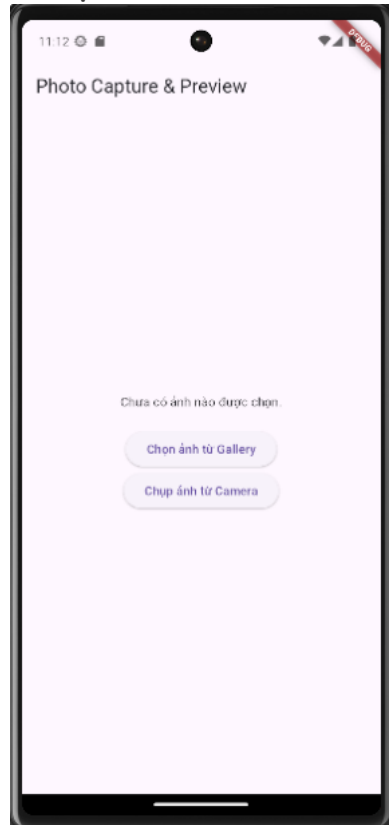
// Widget xem trước toàn màn hình
class FullScreenImage extends StatelessWidget {
  final File imageFile;

  FullScreenImage({required this.imageFile});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Xem trước')),
      body: Center(
        child: Image.file(imageFile),
      ),
    );
  }
}

```

- Màn hình hiển thị:



Màn hình chính



Xem ảnh lấy từ gallery



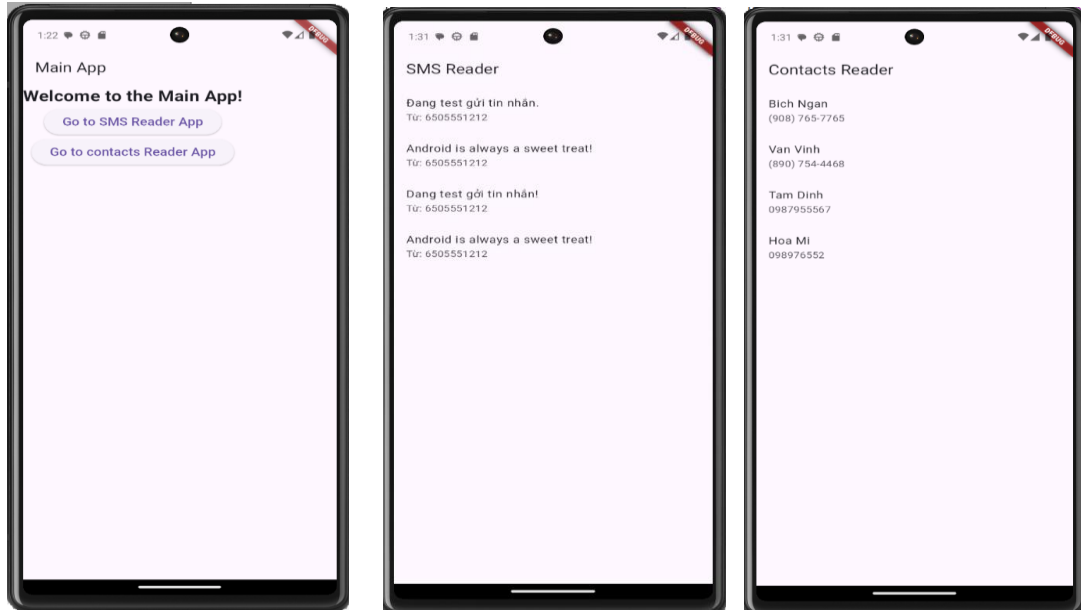
Press Alt to move camera.

Điều khiển camera để chụp
(giả lập trên máy ảo)



Xem ảnh chụp từ camera

Bài tập 3. Xây dựng ứng dụng cho phép hiển thị danh bạ trên điện thoại và đọc, hiển thị các tin nhắn SMS trên thiết bị android.



Hướng dẫn:

Bước 1: Cài đặt các package:

- Tạo project mới dạng Empty Application.
- Mở file pubspec.yaml và thêm vào dependencies:

```
dependencies:  
  flutter:  
    sdk: flutter  
  permission_handler: ^11.4.0  
  flutter_contacts_service: ^0.0.3  
  another_telephony: ^0.3.0
```

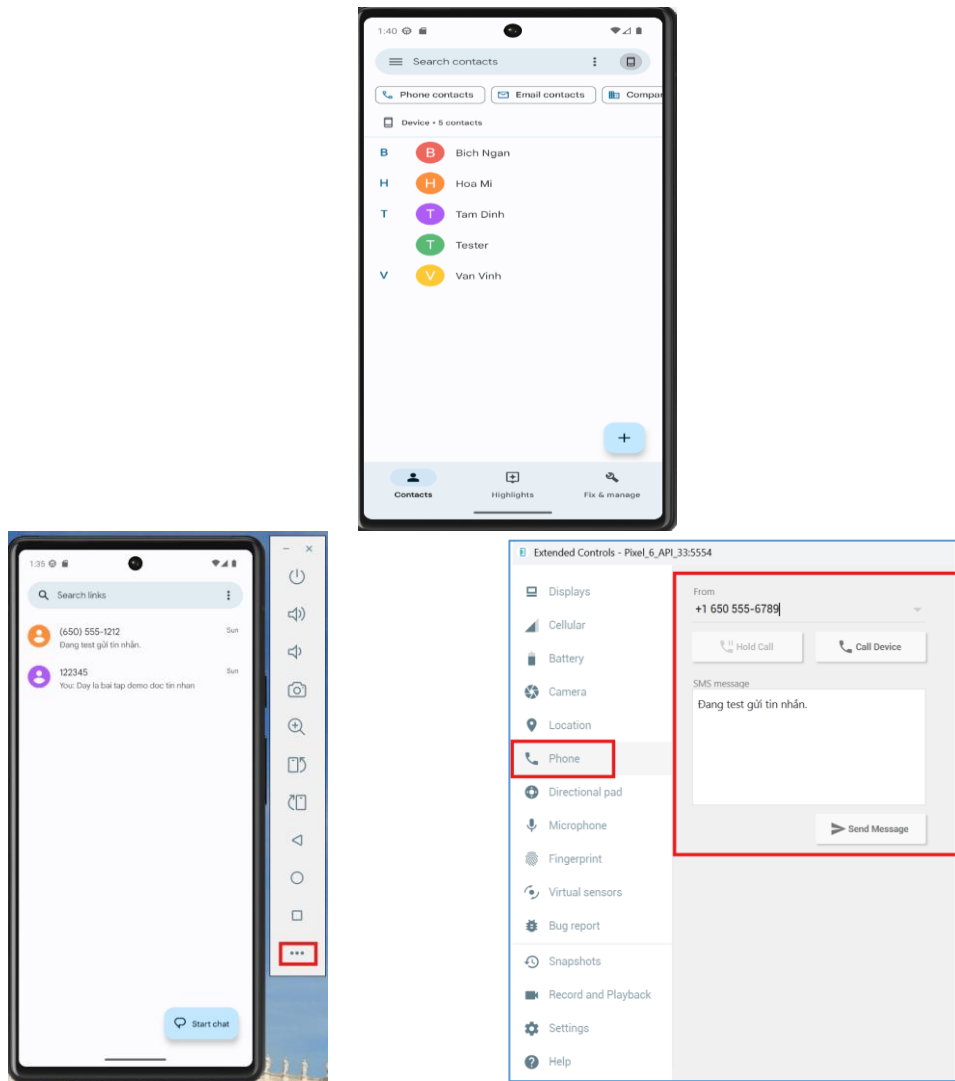
Lưu ý: các version của thư viện có thể update tùy theo thời điểm (dựa vào gợi ý của Copilot để chọn version phù hợp)

- Thêm các quyền truy cập trong file: *android/app/src/main/AndroidManifest.xml*

```
<uses-permission android:name="android.permission.READ_CONTACTS"/>  
<uses-permission android:name="android.permission.WRITE_CONTACTS"/>  
<uses-permission android:name="android.permission.READ_SMS"/>  
<uses-permission android:name="android.permission.RECEIVE_SMS"/>
```

Bước 2. Tạo dữ liệu danh bạ và tin nhắn trên máy ảo để kiểm thử ứng dụng.

- Gửi tin nhắn đến điện thoại ảo : vào phần Extended Controls của máy ảo, chọn mục Phone, chọn số điện thoại và gõ nội dung tin nhắn sms gửi đến máy ảo.
- Vào ứng dụng Contacts thêm 1 số thông tin danh bạ cho máy ảo.



Bước 3. Tạo file *lib/main.dart*

```
import 'package:flutter/material.dart';
import 'package:another_telephony/telephony.dart';
import 'package:permission_handler/permission_handler.dart';
import 'sms_reader_app.dart';
import 'contacts_reader_app.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Main App',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const MainHomePage(),
    );
  }
}
```

```

}

class MainHomePage extends StatelessWidget {
  const MainHomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Main App')),
      body: Column(
        mainAxisAlignment: MainAxisAlignment.start,
        children: [
          const Text(
            'Welcome to the Main App!',
            style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
          ),
          ElevatedButton(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(builder: (context) => const SmsReaderApp()),
              );
            },
            child: const Text(
              'Go to SMS Reader App',
              style: TextStyle(fontSize: 18),
            ),
          ),
          ElevatedButton(
            onPressed: () {
              Navigator.push(
                context,
                MaterialPageRoute(
                  builder: (context) => const ContactsReaderApp(),
                ),
              );
            },
            child: const Text(
              'Go to contacts Reader App',
              style: TextStyle(fontSize: 18),
            ),
          ),
        ],
      ),
    );
  }
}

```

– **Tạo file lib/contacts_reader_app.dart**

```

import 'package:flutter/material.dart';
import 'package:flutter_contacts_service/flutter_contacts_service.dart';

```

```

import 'package:permission_handler/permission_handler.dart';

class ContactsReaderApp extends StatelessWidget {
  const ContactsReaderApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Contacts Reader',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const ContactsReaderHome(),
    );
  }
}

class ContactsReaderHome extends StatefulWidget {
  const ContactsReaderHome({super.key});

  @override
  State<ContactsReaderHome> createState() => _ContactsReaderHomeState();
}

class _ContactsReaderHomeState extends State<ContactsReaderHome> {
  List<ContactInfo> _contacts = [];
  bool _isLoading = true;

  @override
  void initState() {
    super.initState();
    _initializePermissions();
  }

  Future<void> _initializePermissions() async {
    // Yêu cầu quyền truy cập danh bạ
    Map<Permission, PermissionStatus> statuses =
      await [Permission.contacts].request();

    if (statuses[Permission.contacts]!.isGranted) {
      _loadContacts();
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Vui lòng cấp quyền để đọc danh bạ!')),
      );
    }
  }

  Future<void> _loadContacts() async {
    setState(() {
      _isLoading = true;
    });

    // Lấy danh bạ

```

```

List<ContactInfo> contacts = await FlutterContactsService.getContacts();

setState(() {
  _contacts = contacts;
  _isLoading = false;
});
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Contacts Reader')),
    body:
      _isLoading
        ? const Center(child: CircularProgressIndicator())
        : _contacts.isEmpty
        ? const Center(child: Text('Không có danh bạ nào.'))
        : ListView.builder(
            itemCount: _contacts.length,
            itemBuilder: (context, index) {
              ContactInfo contact = _contacts[index];
              return ListTile(
                title: Text(contact.displayName ?? 'Không có tên'),
                subtitle: Text(
                  contact.phones!.isNotEmpty
                    ? contact.phones?.first.value ?? 'Không có số'
                    : 'Không có số',
                ),
              ),
            ),
          );
    ),
  );
}
}

```

– **Tạo file lib/sms_reader_app.dart**

```

import 'package:flutter/material.dart';
import 'package:another_telephony/telephony.dart';
import 'package:permission_handler/permission_handler.dart';

class SmsReaderApp extends StatelessWidget {
  const SmsReaderApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'SMS Reader',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const SmsReaderHome(),
    );
  }
}

```

```

}

class SmsReaderHome extends StatefulWidget {
  const SmsReaderHome({super.key});

  @override
  State<SmsReaderHome> createState() => _SmsReaderHomeState();
}

class _SmsReaderHomeState extends State<SmsReaderHome> {
  final Telephony telephony = Telephony.instance;
  List<SmsMessage> _messages = [];
  bool _isLoading = true;

  @override
  void initState() {
    super.initState();
    _initializePermissions();
  }

  Future<void> _initializePermissions() async {
    // Yêu cầu quyền SMS
    Map<Permission, PermissionStatus> statuses =
      await [Permission.sms, Permission.phone].request();

    if (statuses[Permission.sms]!.isGranted) {
      _loadMessages();
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Vui lòng cấp quyền để đọc tin nhắn SMS!'),
        ),
      );
    }
  }

  Future<void> _loadMessages() async {
    setState(() {
      _isLoading = true;
    });

    // Lấy tin nhắn từ hộp thư đến
    List<SmsMessage> messages = await telephony.getInboxSms(
      columns: [
        SmsColumn.ADDRESS,
        SmsColumn.BODY,
        SmsColumn.DATE,
        SmsColumn.TYPE,
      ],
      sortOrder: [OrderBy(SmsColumn.DATE, sort: Sort.DESC)],
    );
  }
}

```

```

    setState(() {
      _messages = messages;
      _isLoading = false;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('SMS Reader')),
      body:
        _isLoading
        ? const Center(child: CircularProgressIndicator())
        : _messages.isEmpty
        ? const Center(child: Text('Không có tin nhắn nào.'))
        : ListView.builder(
            itemCount: _messages.length,
            itemBuilder: (context, index) {
              SmsMessage message = _messages[index];
              return ListTile(
                title: Text(message.body ?? 'Không có nội dung'),
                subtitle: Text('Từ: ${message.address ?? 'Không rõ'}'),
              );
            },
          ),
    );
  }
}

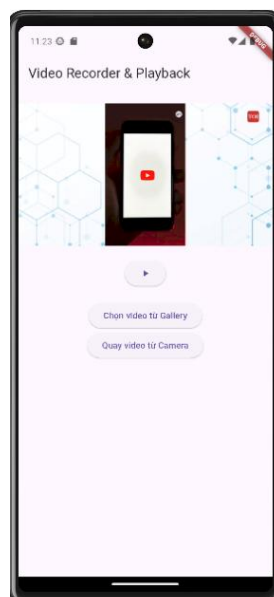
```

3. Bài tập tại lớp

Bài tập 4: Tạo ứng dụng "Video Recorder & Playback", thực hiện các chức năng:

- Quay video từ camera hoặc chọn video từ gallery.
- Phát video với nút play/pause.

Gợi ý: Thực hiện các bước cài đặt và khai báo cấp quyền như bài 1,2.



Phát/dùng video chọn từ gallery

- Mã code :

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import 'package:permission_handler/permission_handler.dart';
import 'package:video_player/video_player.dart';

void main() {
  runApp(VideoRecorderApp());
}

class VideoRecorderApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Video Recorder & Playback',
      theme: ThemeData(primarySwatch: Colors.purple),
      home: VideoRecorderHome(),
    );
  }
}

class VideoRecorderHome extends StatefulWidget {
  @override
  _VideoRecorderHomeState createState() => _VideoRecorderHomeState();
}

class _VideoRecorderHomeState extends State<VideoRecorderHome> {
  File? _videoFile;
  VideoPlayerController? _videoController;
  final ImagePicker _picker = ImagePicker();

  // Yêu cầu quyền
  Future<void> _requestPermission(Permission permission) async {
    if (await permission.isDenied) {
      await permission.request();
    }
  }

  // Chọn video từ gallery
  Future<void> _pickVideoFromGallery() async {
    await _requestPermission(Permission.photos);
    final XFile? pickedFile = await _picker.pickVideo(
      source: ImageSource.gallery,
    );
    if (pickedFile != null) {
      _loadVideo(File(pickedFile.path));
    }
  }
}
```

```

// Quay video từ camera
Future<void> _recordVideoFromCamera() async {
  await _requestPermission(Permission.camera);
  await _requestPermission(Permission.microphone);
  final XFile? recordedFile = await _picker.pickVideo(
    source: ImageSource.camera,
  );
  if (recordedFile != null) {
    _loadVideo(File(recordedFile.path));
  }
}

// Tải và khởi tạo video
void _loadVideo(File videoFile) {
  setState(() {
    _videoFile = videoFile;
    _videoController?.dispose();
    _videoController = VideoPlayerController.file(_videoFile!)
      ..initialize().then((_) {
        setState(() {});
        _videoController!.play();
      });
  });
}

@override
void dispose() {
  _videoController?.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text('Video Recorder & Playback')),
    body: SingleChildScrollView(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          SizedBox(height: 20),
          _videoController != null && _videoController!.value.isInitialized
            ? AspectRatio(
                aspectRatio: _videoController!.value.aspectRatio,
                child: VideoPlayer(_videoController!),
              )
            : Container(
                height: 200,
                child: Center(child: Text('Chưa có video nào được chọn.')),
              ),
          SizedBox(height: 20),
          if (_videoController != null)
            Row(

```

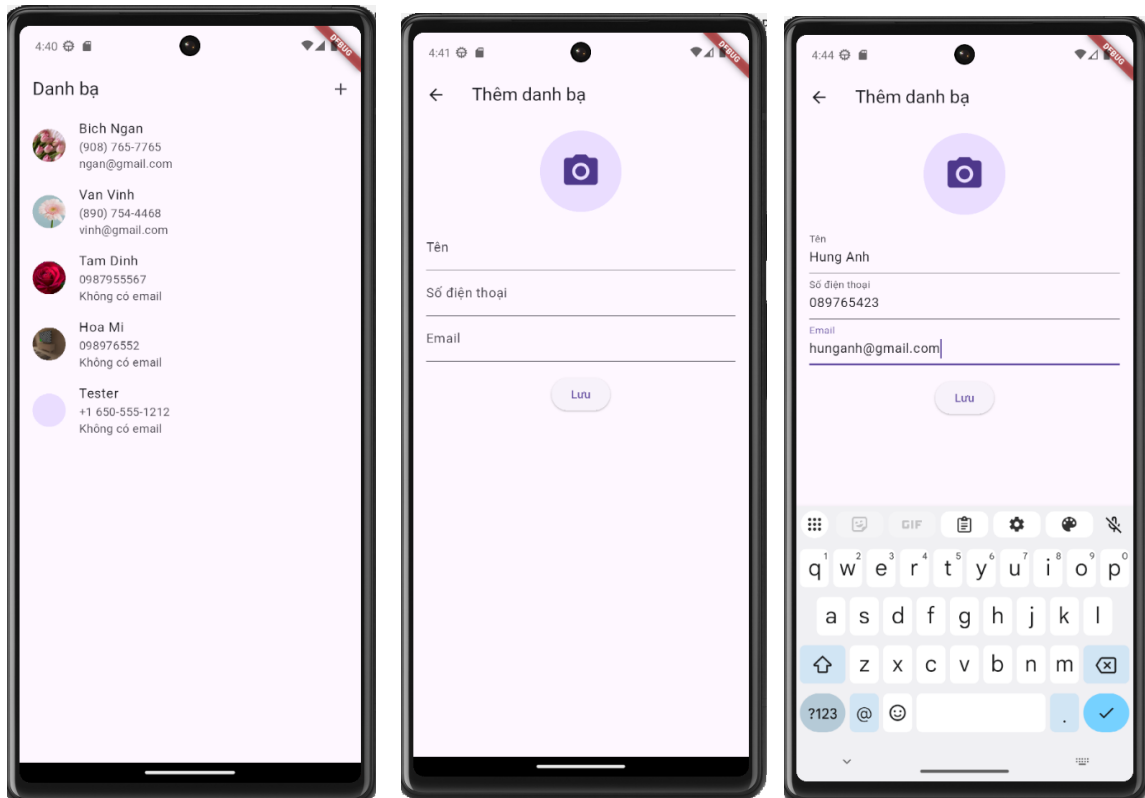
```

        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          ElevatedButton(
            onPressed: () {
              setState(() {
                _videoController!.value.isPlaying
                  ? _videoController!.pause()
                  : _videoController!.play();
              });
            },
            child: Icon(
              _videoController!.value.isPlaying
                ? Icons.pause
                : Icons.play_arrow,
            ),
          ),
          SizedBox(height: 20),
          ElevatedButton(
            onPressed: _pickVideoFromGallery,
            child: Text('Chọn video từ Gallery'),
          ),
          ElevatedButton(
            onPressed: _recordVideoFromCamera,
            child: Text('Quay video từ Camera'),
          ),
        ],
      ),
    ),
  );
}
}

```

Bài tập 5. Tạo ứng dụng quản lý và thêm thông tin danh bạ, bao gồm:

- Màn hình chính hiển thị danh sách danh bạ gồm:
 - Thông tin lấy từ danh bạ của điện thoại
 - Thông tin người dùng tự thêm vào
 - Click vào nút + trên app bar sẽ mở màn hình Thêm danh bạ.
- Màn hình thêm danh bạ:
 Cho phép nhập thông tin danh bạ và thêm ảnh đại diện lấy từ gallery hoặc camera



Gợi ý source code:

- Bổ sung package cần thiết:

```
dependencies:
  flutter:
    sdk: flutter
  permission_handler: ^11.4.0
  flutter_contacts_service: ^0.0.3
  another_telephony: ^0.3.0
  image_picker: ^0.8.7+4
```

- Tạo file `add_contact_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:flutter_contacts_service/flutter_contacts_service.dart';
import 'package:image_picker/image_picker.dart';
import 'dart:io';

class AddContactScreen extends StatefulWidget {
  const AddContactScreen({super.key});

  @override
  State<AddContactScreen> createState() => _AddContactScreenState();
}

class _AddContactScreenState extends State<AddContactScreen> {
  final TextEditingController _nameController = TextEditingController();
  final TextEditingController _phoneController = TextEditingController();
  final TextEditingController _emailController = TextEditingController();
  File? _avatar;

  Future<void> _pickImage(ImageSource source) async {
```

```

final picker = ImagePicker();
final pickedFile = await picker.pickImage(source: source);
if (pickedFile != null) {
  setState(() {
    _avatar = File(pickedFile.path);
  });
}
}

Future<void> _saveContact() async {
  // Kiểm tra nếu tên hoặc số điện thoại bị bỏ trống
  if (_nameController.text.isEmpty || _phoneController.text.isEmpty) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Tên và số điện thoại không được để trống!'),
      ),
    );
    return;
  }

  // Tạo đối tượng ContactInfo
  final contact = ContactInfo(
    displayName: _nameController.text,
    phones: [ValueItem(label: 'mobile', value: _phoneController.text)],
    emails: [ValueItem(label: 'email', value: _emailController.text)],
    avatar: _avatar != null ? await _avatar!.readAsBytes() : null,
  );

  // Lưu danh bạ vào điện thoại
  try {
    await FlutterContactsService.addContact(contact);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Danh bạ đã được lưu thành công!')),
    );
    Navigator.pop(context); // Quay lại màn hình trước
  } catch (e) {
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text('Lỗi khi lưu danh bạ: $e')));
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Thêm danh bạ')),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        children: [
          GestureDetector(
            onTap: () => _pickImage(ImageSource.gallery),
            child: CircleAvatar(

```

```

        radius: 50,
        backgroundImage: _avatar != null ? AssetImage(_avatar!) : null,
        child:
            _avatar == null
                ? const Icon(Icons.camera_alt, size: 50)
                : null,
    ),
),
const SizedBox(height: 16),
TextField(
    controller: _nameController,
    decoration: const InputDecoration(labelText: 'Tên'),
),
TextField(
    controller: _phoneController,
    decoration: const InputDecoration(labelText: 'Số điện thoại'),
    keyboardType: TextInputType.phone,
),
TextField(
    controller: _emailController,
    decoration: const InputDecoration(labelText: 'Email'),
    keyboardType: TextInputType.emailAddress,
),
const SizedBox(height: 16),
ElevatedButton(onPressed: _saveContact, child: const Text('Lưu')),
],
),
),
);
}
}

```

- Tạo file *contacts_list_screen.dart*

```

import 'package:flutter/material.dart';
import 'package:flutter_contacts_service/flutter_contacts_service.dart';
import 'add_contact_screen.dart';

class ContactsListScreen extends StatefulWidget {
    const ContactsListScreen({super.key});

    @override
    State<ContactsListScreen> createState() => _ContactsListScreenState();
}

class _ContactsListScreenState extends State<ContactsListScreen> {
    List<ContactInfo> _contacts = [];
    bool _isLoading = true;

    @override
    void initState() {

```

```

    super.initState();
    _loadContacts();
}

Future<void> _loadContacts() async {
  setState(() {
    _isLoading = true;
  });

  List<ContactInfo> contacts = await FlutterContactsService.getContacts();
  setState(() {
    _contacts = contacts;
    _isLoading = false;
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Danh bạ'),
      actions: [
        IconButton(
          icon: const Icon(Icons.add),
          onPressed: () async {
            await Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => const AddContactScreen(),
              ),
            );
            _loadContacts(); // Refresh danh bạ sau khi thêm
          },
        ),
      ],
    ),
    body:
      _isLoading
        ? const Center(child: CircularProgressIndicator())
        : _contacts.isEmpty
        ? const Center(child: Text('Không có danh bạ nào.'))
        : ListView.builder(
            itemCount: _contacts.length,
            itemBuilder: (context, index) {
              ContactInfo contact = _contacts[index];
              return ListTile(
                leading:
                  contact.avatar != null
                    ? CircleAvatar(
                        backgroundImage: MemoryImage(contact.avatar!),
                      )
                    : const CircleAvatar(child: Icon(Icons.person)),
              );
            },
          );
}

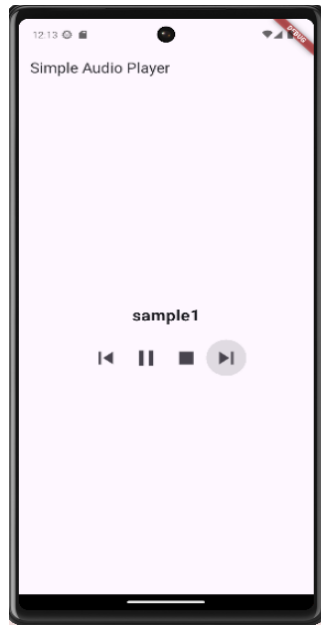
```

```

        title: Text(contact.displayName ?? 'Không có tên'),
        subtitle: Text(
            '${(contact.phones != null &&
contact.phones!.isEmpty) ? contact.phones!.first.value ?? 'Không có số' :
'Không có số'}\n'
            '${(contact.emails != null &&
contact.emails!.isEmpty) ? contact.emails!.first.value ?? 'Không có email'
: 'Không có email'}',
        ),
    );
},
),
);
}
}
}

```

Bài tập 6. Tọa ứng dụng cho phép phát file audio, gồm : Play, Pause, Stop, Next, và Previous.



Gợi ý:

- Sử dụng package : **audioplayers**
- Tạo thư mục assets/audios trong thư mục gốc của dự án.
- Thêm một số file audio (ví dụ: song1.mp3, song2.mp3, song3.mp3) vào thư mục này.
- Cập nhật file pubspec.yaml để khai báo assets:

```
flutter:  
  assets:  
    - assets/audios/
```

- Cài đặt package:

```
dependencies:
  flutter:
    sdk: flutter
  audioplayers: ^5.2.1
```

- Mã code:

```

import 'package:flutter/material.dart';
import 'package:audioplayers/audioplayers.dart';

void main() {
  runApp(SimpleAudioPlayer());
}

class SimpleAudioPlayer extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Simple Audio Player',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: AudioPlayerHome(),
    );
  }
}

class AudioPlayerHome extends StatefulWidget {
  @override
  _AudioPlayerHomeState createState() => _AudioPlayerHomeState();
}

class _AudioPlayerHomeState extends State<AudioPlayerHome> {
  late AudioPlayer _audioPlayer;
  int _currentSongIndex = 0;
  bool _isPlaying = false;

  // Danh sách các bài hát (từ assets)
  final List<String> _songs = [
    'assets/audios/sample1.mp3',
    'assets/audios/sample2.mp3',
    'assets/audios/sample3.mp3',
  ];

  // Tên bài hát để hiển thị
  final List<String> _songTitles = ['sample1', 'sample2', 'sample3'];

  @override
  void initState() {
    super.initState();
    _audioPlayer = AudioPlayer();

    // Lắng nghe trạng thái phát
    _audioPlayer.onPlayerStateChanged.listen((PlayerState state) {
      setState(() {
        _isPlaying = state == PlayerState.playing;
      });
    });

    // Lắng nghe khi bài hát kết thúc
    _audioPlayer.onPlayerComplete.listen((event) {
      _nextSong(); // Tự động chuyển bài khi hết
    });
  }
}

```

```

}

@override
void dispose() {
  _audioPlayer.dispose();
  super.dispose();
}

// Phát bài hát
Future<void> _playSong() async {
  await _audioPlayer.play(
    AssetSource(_songs[_currentSongIndex].replaceAll('assets/', '')),
  );
  setState(() {
    _isPlaying = true;
  });
}

// Tạm dừng bài hát
Future<void> _pauseSong() async {
  await _audioPlayer.pause();
  setState(() {
    _isPlaying = false;
  });
}

// Dừng bài hát
Future<void> _stopSong() async {
  await _audioPlayer.stop();
  setState(() {
    _isPlaying = false;
  });
}

// Chuyển sang bài tiếp theo
void _nextSong() {
  setState(() {
    if (_currentSongIndex < _songs.length - 1) {
      _currentSongIndex++;
    } else {
      _currentSongIndex = 0; // Quay lại bài đầu nếu hết danh sách
    }
    _stopSong(); // Dừng bài hiện tại trước khi phát bài mới
    _playSong();
  });
}

// Quay lại bài trước
void _previousSong() {
  setState(() {
    if (_currentSongIndex > 0) {
      _currentSongIndex--;
    }
  });
}

```

```

    } else {
      _currentSongIndex =
        _songs.length - 1; // Chuyển đến bài cuối nếu đang ở đầu
    }
    _stopSong(); // Dừng bài hiện tại trước khi phát bài mới
    _playSong();
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text('Simple Audio Player')),
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          // Hiển thị tên bài hát
          Text(
            _songTitles[_currentSongIndex],
            style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
          ),
          SizedBox(height: 20),
          // Nút điều khiển
          Row(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              IconButton(
                icon: Icon(Icons.skip_previous, size: 40),
                onPressed: _previousSong,
              ),
              IconButton(
                icon: Icon(
                  _isPlaying ? Icons.pause : Icons.play_arrow,
                  size: 40,
                ),
                onPressed: () {
                  if (_isPlaying) {
                    _pauseSong();
                  } else {
                    _playSong();
                  }
                },
              ),
              IconButton(
                icon: Icon(Icons.stop, size: 40),
                onPressed: _stopSong,
              ),
              IconButton(
                icon: Icon(Icons.skip_next, size: 40),
                onPressed: _nextSong,
              ),
            ],
          ),
        ],
      ),
    ),
  );
}

```

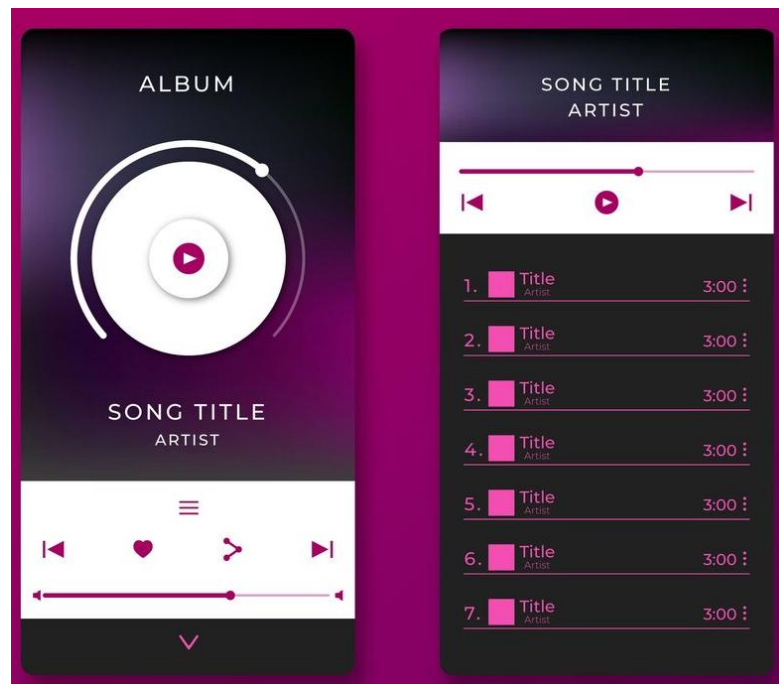
```

        SizedBox(height: 20),
      ],
    ),
  ],
),
),
);
}
}

```

4. Bài tập về nhà

Bài tập 6. Xây dựng ứng dụng nghe nhạc có các chức năng như ảnh, SV tự do thiết kế giao diện



Bài tập 7. Tạo ứng dụng SMS Analyzer thực hiện các chức năng sau :

- Lấy thông tin tất cả tin nhắn của điện thoại, hiển thị thống kê :
 - + Tổng số tin nhắn nhận được.
 - + Danh sách tin nhắn từ một số cụ thể (lọc theo số điện thoại).
 - + Thống kê tin nhắn theo ngày/tháng
- Lọc các tin nhắn theo nhóm :
 - + quảng cáo : có chuỗi [QC] đứng đầu tin nhắn
 - + mã OTP : có chuỗi [OTP] + [chuỗi 6 ký số]
- Đối với các tin nhắn OPT, khi được chọn thì xử lý lấy chuỗi 6 ký số của OTP và hiển thị